

Projeto Final em Engenharia Informática

Emulação de Pedal Analógico com Machine Learning

BLACK-BOX MODELING DO DARKGLASS ALPHA OMICRON

RELATÓRIO FINAL

Nuno Rodrigues · 2201022

Orientador: Pedro Pestana

6 de Maio de 2026

Índice

Capítulo 1 - Introdução.....	6
Capítulo 2 - Desenho.....	6
2.1. Tópico.....	6
2.1.1. Sub-tópico.....	6
Capítulo 3 - Implementação.....	7
Capítulo 4 - Testes.....	7
Capítulo 5 - Conclusões.....	7
Bibliografia.....	7
Anexo I - xxx.....	8

Lista de Figuras

<Inserir índice de figuras>

Lista de Tabelas

<Inserir índice de tabelas>

Resumo

Este projeto desenvolve um *plugin* de áudio nos formatos VST3, AU e Standalone que reproduz o comportamento sonoro do pedal de distorção analógico Darkglass Alpha Omicron, recorrendo a uma rede neuronal recorrente (LSTM) treinada por *black-box modeling* sobre pares de áudio *Dry/Wet* capturados do hardware original. A solução articula três subsistemas: um *pipeline* de treino em Python/PyTorch, um *plugin* nativo C++ construído com JUCE e RTNeural para inferência em tempo real na *audio thread*, e uma WebApp *client-side* (HTML/CSS/JS) que implementa um teste ABX duplo-cego para validação perceptiva.

À data de submissão deste relatório intercalar (Semana 8 de 16), o caminho crítico do projeto está concluído: o *plugin* opera em tempo real numa DAW com um modelo treinado em dados reais a apresentar $ESR \approx 0,030$ (3% de erro relativo); a WebApp ABX está implementada com motor de 12 *trials*, persistência *client-side* e análise estatística.

Capítulo 1 - Introdução

Este capítulo introduz o projeto final de licenciatura, contextualizando-o e enquadrando-o no domínio do desenvolvimento de software de áudio profissional. Começa por uma descrição do trabalho, seguida do âmbito e do enquadramento que o motivam. São identificadas as necessidades que o projeto visa colmatar, os objetivos delineados, as atividades planeadas para os atingir e as restrições a observar. O capítulo termina com a apresentação dos resultados esperados, do calendário global de execução e da estrutura adotada para o presente relatório.

1.1 Descrição do Projeto

O projeto consiste no desenvolvimento de um *plugin* de áudio, nos formatos VST3, AU e Standalone, que reproduz o comportamento sonoro do pedal de distorção analógico **Darkglass Alpha Omicron** - um equipamento de referência no nicho do baixo elétrico - recorrendo a Inteligência Artificial em substituição do Processamento Digital de Sinal (DSP) clássico.

A solução articula três componentes complementares. O primeiro é o **modelo de Inteligência Artificial**: uma rede neuronal treinada em PyTorch que aprende a impressão digital acústica do pedal a partir de pares de sinais *Dry/Wet* recolhidos numa sessão de *reamping*. Esta abordagem, conhecida por *black-box modeling*, dispensa a modelação eletrotécnica do circuito, inferindo a sua resposta diretamente dos dados. O segundo é o **plugin nativo em C++**, construído com o *framework* JUCE e tendo como núcleo de inferência a biblioteca RTNeural, otimizada para execução determinística na *audio thread* sob restrições rigorosas de latência. O terceiro é uma **aplicação web de validação ABX**, que permite testar de forma duplo-cega se ouvintes treinados conseguem distinguir o áudio gerado pelo *plugin* do áudio captado a partir do pedal físico, traduzindo o resultado em evidência estatística (teste binomial e d' da Teoria da Detecção de Sinal).

1.2 Âmbito e Enquadramento

O projeto enquadra-se na unidade curricular de Projeto Final da Licenciatura em Engenharia Informática e situa-se na interseção de três domínios técnicos: **processamento de áudio em tempo real, aprendizagem automática aplicada e desenvolvimento web**. Mobiliza, por isso, conhecimentos transversais ao plano curricular como programação em C++ e Python, engenharia de *software*, sistemas, interface humano-máquina e tecnologias web.

Em termos de domínio aplicacional, o projeto insere-se na tendência recente de utilização de redes neurais para emulação de *hardware* de áudio analógico (amplificadores de

guitarra, pedais de efeitos, processadores de estúdio), uma área em rápida expansão na indústria musical e que tem mostrado resultados perceptivos superiores aos dos métodos de DSP tradicionais quando o circuito modelado apresenta forte não-linearidade - caso típico dos pedais de distorção.

1.3 Necessidades

O projeto visa responder a um conjunto de necessidades concretas:

- **Acesso ao timbre de equipamento dispendioso ou indisponível.** O Darkglass Alpha Omicron é um pedal de gama média-alta, cujo custo e disponibilidade limitam o seu uso a uma franja restrita de utilizadores. Uma emulação fiel democratiza o acesso ao seu carácter sonoro.
- **Integração no fluxo de trabalho moderno.** A produção musical contemporânea decorre maioritariamente em *Digital Audio Workstations* (DAW), onde se valoriza a recuperação total de sessões, a automação de parâmetros e a ausência de *latência* perceptível. Um *plugin* nativo cumpre estes requisitos; o equipamento físico, não.
- **Limitações das emulações tradicionais.** As emulações por DSP clássico tendem a aproximar a resposta do circuito por filtros e funções de saturação fixas, perdendo nuances do comportamento dinâmico do equipamento original. As redes neuronais, treinadas sobre dados reais, têm potencial para capturar essas nuances com maior fidelidade.

1.4 Objetivos

Os objetivos do projeto, derivados das necessidades acima, são:

- **Construir um *plugin* VST3/AU funcional**, instanciável em qualquer DAW compatível, com controlos de *Bypass* e *Output Volume* operacionais e estáveis.
- **Treinar e integrar um modelo de rede neuronal** capaz de processar áudio em tempo real, respeitando os limites de latência típicos de um buffer de 64–512 amostras a 44,1 kHz, sem *glitches* nem sobre-utilização de CPU.
- **Desenvolver uma aplicação web ABX duplo-cega**, acessível por URL público, que recolha respostas de múltiplos participantes em simultâneo e as persista para análise posterior.
- **Avaliar a qualidade da emulação** em duas dimensões: técnica (cumprimento dos requisitos de latência e estabilidade) e percetiva (resultado estatístico do teste ABX).

1.5 Trabalho a Desenvolver

Para alcançar os objetivos delineados, o projeto organiza-se em cinco frentes de trabalho:

- **Aquisição de dados.** Realização de uma sessão de *reamping* com o pedal físico, na qual se reproduzem sinais DI conhecidos de baixo elétrico através do equipamento e se captam as respostas processadas, gerando um corpus de pares *Dry/Wet* que servirá de base ao treino. Esta fase inclui o pré-processamento dos pares (normalização, segmentação, *data augmentation*) e a sua divisão em conjuntos de treino, validação e teste.
- **Treino do modelo.** Implementação, em PyTorch, de uma arquitetura de rede neuronal adequada ao problema (tipicamente uma rede recorrente leve ou convolucional causal) e respetivo pipeline de treino. O modelo é exportado num formato consumível pela RTNeural.
- **Desenvolvimento do *plugin*.** Construção, em C++ com JUCE, da infraestrutura do *plugin*: gestão de I/O, *bypass*, ganho de saída, interface gráfica e ciclo de vida na DAW. Integração da RTNeural na *audio thread* para inferência sem alocações dinâmicas de memória.
- **Aplicação Web ABX.** Implementação de um cliente HTML/CSS/JavaScript que apresenta amostras A, B e X aleatorizadas, recolhe a resposta do participante e a submete a um *backend* mínimo de persistência.
- **Avaliação e análise.** Verificação automatizada dos critérios técnicos (latência, ausência de *dropouts*) e análise estatística dos resultados do teste ABX, recorrendo ao teste binomial e ao cálculo do d' da Teoria da Detecção de Sinal.

1.6 Restrições

O projeto desenvolve-se sob restrições que condicionam decisões de arquitetura e de planeamento:

- **Tempo real.** A inferência ocorre na *audio thread*, pelo que não é admissível nenhum bloqueio, alocação dinâmica de memória ou operação de I/O durante o processamento de um bloco.
- **Compatibilidade multiplataforma.** Os formatos VST3 e AU exigem conformidade com especificações distintas e testes em mais do que um sistema operativo (Linux, Windows e macOS).
- **Disponibilidade de *hardware*.** A captura *Dry/Wet* depende de acesso ao pedal Darkglass Alpha Omicron e a uma cadeia de *reamping* correta.

- **Duração letiva.** O projeto está enquadrado num calendário académico de aproximadamente 16 semanas, o que impõe disciplina de escopo e priorização rigorosa do MVP.
- **Plano de contingência.** Em fase de proposta foi previsto que, caso a inferência por IA apresentasse latência excessiva ou instabilidade não resolúvel até à demonstração interna, a componente de IA seria substituída por um algoritmo de distorção *open-source*. À data do presente relatório, esta contingência **não foi acionada**: a inferência em tempo real opera de forma estável com o modelo treinado integrado no *plugin*, pelo que o caminho original com IA é mantido.

1.7 Resultados Esperados

Após a conclusão do projeto, esperam-se os seguintes resultados:

- **Um *plugin* distribuível** (binários VST3, AU e Standalone) que opere em hosts compatíveis sem regressões funcionais.
- **Um modelo treinado**, juntamente com o pipeline de treino, exportado em formato reproduzível.
- **Uma WebApp ABX publicada** num URL acessível ao público, capaz de recolher e armazenar respostas de participantes voluntários.
- **Evidência estatística**, sob a forma de relatório, sobre o grau de indistinguilidade percetiva entre o *plugin* e o pedal físico, traduzida em *p-value* (teste binomial) e *d'*.
- **Documentação técnica** que permita a um terceiro reproduzir, ampliar ou auditar o trabalho.

1.8 Calendário Geral

O cronograma global do projeto, conforme aprovado em fase de proposta, está organizado em 16 semanas distribuídas por sete fases:

Fase	Sem.	Período	Marcos principais
Arranque e Scaffolding	1–2	17–28 Mar	Configuração JUCE; boilerplate do plugin; submissão da Proposta
Requisitos e Dados	3–4	31 Mar–11 Abr	MoSCoW; diagramas C4; setup GitHub; sessão de reamping
Treino do Modelo	5–6	14–25 Abr	Pipeline PyTorch; ADRs; primeiros testes de inferência

Demo Interna	7	28 Abr–2 Mai	Demonstração ao orientador; decisão IA vs. fallback DSP
Relatório Intercalar	8	5–6 Mai	Entrega do presente relatório
Otimização e WebApp	9–10	7–16 Mai	Quantização/pruning; profiling de CPU; WebApp ABX
Integração e Testes	11–12	19–30 Mai	MVP completo; lançamento público do teste ABX
Análise e Fecho	13	2–6 Jun	Análise estatística; validação dos critérios
Redação Final	14	9–13 Jun	Capítulos finais; formatação APA; anexos
Revisão e Defesa	15	16–20 Jun	Reunião com orientador; revisão final
Submissão Final	16	24 Jun	Entrega do relatório, código-fonte e demo funcional

À data do presente relatório, o projeto encontra-se na **Semana 8**. As fases de Arranque/*Scaffolding*, Requisitos e Dados, Treino do Modelo e Demonstração Interna estão concluídas; o *plugin* opera em tempo real numa DAW com o modelo treinado integrado.

1.9 Estrutura do Relatório

O relatório final está organizado em cinco capítulos, complementados por bibliografia e anexos:

- **Capítulo 1 - Introdução.** Apresenta o âmbito, o enquadramento, as necessidades, os objetivos, as restrições, os resultados esperados, o calendário e a estrutura do documento.
- **Capítulo 2 - Desenho.** Identifica e justifica as tecnologias adotadas (*hardware, software, linguagens e frameworks*), descreve a arquitetura do sistema e dos seus principais componentes, define a estrutura de dados e os algoritmos centrais, fundamenta as opções tomadas face às alternativas consideradas e apresenta o desenho da interface do utilizador, recorrendo a diagramas UML e C4 para complementar a descrição.
- **Capítulo 3 - Implementação.** Decompõe o trabalho em tarefas, identifica precedências, apresenta o calendário detalhado de execução e reporta o estado de cada tarefa.

- **Capítulo 4 - Testes.** Documenta a metodologia de validação técnica e percetiva, os testes de funcionalidade, desempenho e escalabilidade, e os respetivos resultados.
- **Capítulo 5 - Conclusões.** Reflete sobre os resultados face aos objetivos, identifica dificuldades, limitações e possíveis melhorias.

Capítulo 2 - Desenho

Este capítulo apresenta o desenho da solução técnica adotada para o projeto, justificando as escolhas tomadas face às alternativas ponderadas. Começa por uma visão geral da arquitetura e enumera as tecnologias e ferramentas utilizadas em cada componente. Apresenta em seguida a arquitetura do sistema através dos diagramas C4 de nível 1 e 2, expõe os fundamentos conceptuais da abordagem *black-box*, descreve o desenho da rede neuronal e do *pipeline* de treino, formaliza o protocolo de aquisição e alinhamento de dados, especifica o desenho da inferência em tempo real, define a estrutura de dados e classes principais, e termina com a apresentação das interfaces do utilizador e uma síntese das decisões de arquitetura.

2.1 Visão Geral da Arquitetura

A solução é composta por **três subsistemas** com responsabilidades disjuntas:

- **Pipeline de treino** (Python / PyTorch). Consome pares de áudio *Dry/Wet*, treina uma rede neuronal e exporta os pesos num formato neutro (JSON).
- **Plugin de áudio nativo** (C++ / JUCE / RTNeural). Carrega os pesos exportados, instancia a rede em memória e processa áudio em tempo real na *audio thread* da DAW *host*.
- **WebApp de validação ABX** (HTML/CSS/JS). Apresenta a participantes externos pares de excertos de áudio gerados *offline* — uns processados pelo *plugin*, outros pelo *hardware* original — e regista as suas escolhas para análise estatística posterior.

2.2 Tecnologias e Ferramentas

A seleção tecnológica obedece a três critérios transversais: **maturidade**, **adequação a restrições de tempo real** (no subsistema do *plugin*) e **neutralidade de formato** (nas

fronteiras entre subsistemas). A Tabela 2.1 sintetiza as escolhas; as subsecções seguintes detalham as justificações e as alternativas consideradas.

Tabela 2.1 — Stack tecnológica por subsistema

Subsistema	Componente	Tecnologia	Função
Treino	Linguagem	Python 3.11	Script de pipeline
Treino	Framework ML	PyTorch	Definição da rede, autograd, treino
Treino	I/O de áudio	soundfile (libsndfile)	Leitura/escrita de WAV 24 bit
Treino	Pré-processamento	NumPy	Cross-correlation, slicing, normalização
Plugin	Linguagem	C++17	Plugin nativo
Plugin	Framework	JUCE	Abstração VST3/AU, GUI, audio I/O
Plugin	Inferência	RTNeural (backend Eigen)	LSTM/Dense em audio thread
Plugin	Build	CMake + CPM	Resolução de dependências source-level
WebApp	Linguagens	HTML5, CSS3, JavaScript	Página client-side
WebApp	Persistência	supabase	Registo de respostas

2.2.1 Pipeline de Treino

PyTorch foi adotado por ser o ecossistema dominante na investigação académica em redes neurais, o que se traduz em vasta literatura, exemplos de referência e *tooling* maduro. Para o problema concreto — treino de uma LSTM pequena com gradientes calculados sobre janelas de áudio — a *framework* não introduz qualquer restrição relevante.

Alternativas consideradas:

- *TensorFlow / Keras*: igualmente capaz, mas menos representado na literatura específica de emulação neural de áudio (Wright et al., 2019; Damskögg et al., 2019).

soundfile foi preferido a torchaudio para escrita de ficheiros WAV após uma falha encontrada na fase de alinhamento: o *backend* recente do torchaudio.save (TorchCodec) ignorava silenciosamente operações de *slicing*, comprometendo a integridade dos dados. O soundfile opera diretamente sobre libsndfile com comportamento determinístico e suporte garantido para 24 *bit*.

2.2.2 Plugin de Áudio

JUCE é a *framework* C++ de referência para desenvolvimento de *plugins* de áudio multiplataforma. Abstrai a complexidade dos formatos VST3 e AU, fornece APIs de alto nível para gestão da *audio thread* e da interface gráfica, e dispõe de mecanismos idiomáticos para exposição de parâmetros à DAW e à GUI.

Alternativas consideradas:

- Implementação de raiz dos *bindings* VST3/AU: tecnicamente possível, mas o esforço seria desproporcional ao escopo de um projeto de licenciatura, sem benefício técnico discernível.

RTNeural é uma biblioteca C++ desenhada especificamente para inferência de redes neurais em ambiente de áudio em tempo real. As suas características diferenciadoras são:

- ausência de alocações dinâmicas de memória durante o *forward pass*;
- API *compile-time* baseada em *templates* que permite ao compilador gerar código altamente otimizado e *inlinable*;
- *backends* selecionáveis (Eigen, *xsimd*, STL) com Eigen a oferecer melhor desempenho para LSTMs pequenas.

Alternativas consideradas:

- *LibTorch* (PyTorch C++): excelente para inferência *offline*, mas com latência de inicialização e *overhead* de execução incompatíveis com o processamento amostra a amostra.

2.2.3 WebApp ABX

A WebApp adota uma *stack* deliberadamente leve — **HTML, CSS e JavaScript** *client-side*, sem *frameworks* pesados — pelas seguintes razões:

- o problema é estruturalmente simples (apresentação de áudio + recolha de respostas), pelo que React, Vue ou similares introduziriam complexidade desproporcional;
- o *deployment* em serviços estáticos (GitHub Pages, Netlify) é trivial;
- a portabilidade entre navegadores é elevada e a manutenção, mínima.

A WebApp foi desenvolvida com antecipação face ao calendário inicial do projeto e submetida como E-Fólio A da unidade curricular de *Laboratório de Sistemas e Serviços Web*, capitalizando a sobreposição temática.

2.4 Modelação Black-box: Fundamento Conceptual

A decisão arquitetural mais consequente do projeto é a opção pela **modelação black-box** do circuito analógico.

Esta abordagem trata o pedal como uma função desconhecida $f: x(t) \rightarrow y(t)$ e aprende uma aproximação f a partir de pares observados (x, y) , sem qualquer pressuposto sobre a estrutura interna do circuito. Esta opção é justificada por três fatores:

1. **Disponibilidade de dados.** O pedal físico está disponível e pode ser estimulado com sinais conhecidos, gerando pares *Dry/Wet* arbitrariamente longos. A escassez de dados — que tipicamente desfavorece *black-box* — não se aplica.
2. **Complexidade não-linear.** Pedais de distorção combinam saturação não-linear, filtragem dependente de amplitude e dinâmica reativa de condensadores. Modelar este conjunto de efeitos analiticamente é trabalhoso e propenso a erros; modelos *black-box* aprendem-no diretamente da resposta observada.
3. **Estado da arte.** A literatura recente em emulação neural de amplificadores e pedais (Wright et al., 2019; *GuitarML*; *Neural Amp Modeler*) estabelece a abordagem *black-box* como o *state of the art* para esta classe de problemas, com resultados percetivos consistentemente superiores aos das alternativas.

Limitações reconhecidas: o modelo aprendido é específico da configuração do pedal no momento da captura. Se os controlos físicos (Drive, Tone, Level, etc...) forem alterados, é necessário recapturar e voltar a treinar — cada combinação de configurações é, do ponto de vista do modelo, um pedal diferente.

2.5 Arquitetura da Rede Neuronal

2.5.1 Escolha da Arquitetura

Foi adotada uma **rede recorrente do tipo LSTM**. A justificação assenta numa analogia entre arquiteturas de rede e topologias clássicas de filtros digitais:

- Uma **MLP** (*Multilayer Perceptron*) é *memoryless*: a sua saída depende apenas da entrada atual, sem qualquer estado interno. Equivale, conceptualmente, a uma função estática.
- Uma **CNN** (*Convolutional Neural Network*) ou **TCN** (*Temporal Convolutional Network*) é análoga a um **filtro FIR**: a saída depende de uma janela explícita das N amostras de entrada anteriores, sem estado.
- Uma **LSTM** é análoga a um **filtro IIR**: mantém um estado comprimido que recursivamente acumula informação de toda a história passada do sinal.

Os circuitos analógicos com elementos reativos (condensadores, indutores) **têm memória interna**: a sua resposta depende não apenas do sinal atual mas do passado recente, codificado na carga acumulada nos condensadores. Esta memória é compactamente representável por um estado finito — exatamente o que uma LSTM oferece. Uma MLP, sendo *memoryless*, é estruturalmente incapaz de representar este comportamento; uma CNN/TCN consegue, mas à custa de receber explicitamente uma janela longa do passado, o que aumenta o custo computacional na *audio thread*.

A LSTM é, portanto, a escolha **estrutural** para o problema, e é também a opção dominante na literatura de referência.

2.5.2 Topologia

A topologia adotada é minimalista: $\text{input}(1) \rightarrow \text{LSTM}(\text{hidden}=24) \rightarrow \text{Dense}(1)$.

- **Entrada**: 1 canal (a amostra de áudio mono no instante t).
- **Camada LSTM**: 24 neurónios na camada oculta. A escolha do tamanho é discutida na subsecção seguinte.
- **Camada Dense de saída**: 1 neurónio linear, sem ativação final, produzindo a amostra processada.

2.5.3 Dimensionamento

O tamanho da camada oculta é condicionado, **em primeira instância, pelo orçamento computacional da *audio thread***, e só depois pela qualidade percetiva pretendida. A estratégia adotada foi começar por uma LSTM de pequena dimensão (`hidden=8`) e aumentar progressivamente apenas se os critérios de aceitação não fossem satisfeitos. Esta política iterativa garante que qualquer modelo produzido cumpre, à partida, a restrição de tempo real.

2.6 Pipeline de Treino

2.6.1 Estratégia de Janelas

O treino opera sobre **janelas curtas** do sinal, agrupadas em *mini-batches*, em vez de processar o sinal completo por *epoch*. A justificação tem três componentes:

1. **Viabilidade computacional.** *Backpropagation* sobre cinco minutos de áudio a 48 kHz é inviável em memória e numericamente instável (*vanishing gradient*).
2. **Convergência.** Mais janelas por *epoch* significam mais atualizações de parâmetros por unidade de tempo, acelerando a convergência face a uma única atualização por sinal completo.
3. **Robustez a início de sinal.** Como cada janela começa com o estado interno da LSTM a zero (por defeito), a rede é forçada a aprender a reagir bem ao transitório inicial. Tal é desejável: numa DAW, o utilizador pode ativar o *plugin* a meio de uma sessão, e a rede tem de produzir saída coerente desde a primeira amostra.

A **dimensão da janela** é um compromisso documentado. Janelas demasiado curtas — por exemplo, 20 amostras (~0,4 ms a 48 kHz) — não contêm sequer um ciclo completo da frequência fundamental do baixo elétrico (a nota mais grave, Mi 41 Hz, tem período de cerca de 1150 amostras). A LSTM, sem ver um ciclo, fica reduzida a uma função *memoryless* — anulando exatamente a vantagem da arquitetura. Janelas demasiado longas reintroduzem o problema do *vanishing gradient*. O valor adotado é de **2048 amostras** (~43 ms a 48 kHz), correspondente ao *sweet spot* reportado pela literatura (Wright et al., 2019).

2.6.2 Truncated Backpropagation Through Time (TBPTT)

Para mitigar o *vanishing gradient* dentro de cada janela, é aplicada **TBPTT**: cada janela de 2048 amostras é dividida em chunks de 1024 amostras, com o estado interno propagado entre chunks mas **desligado do grafo de gradientes** (`.detach()`) entre eles. Esta técnica

permite à rede aprender dependências temporais até ao limite de um chunk sem que os gradientes propaguem indefinidamente para trás.

2.6.3 Função de Custo: Error-to-Signal Ratio (ESR)

Foi adotada a função de custo **ESR** (*Error-to-Signal Ratio*), conforme proposto em Wright et al. (2019).

Comparativamente ao MSE absoluto, o ESR **normaliza o erro pela energia do sinal *target***. Esta normalização tem dois efeitos:

- impede que o modelo dedique capacidade a aprender variações de ganho global em vez da estrutura não-linear do sistema;
- torna o valor da *loss* diretamente interpretável como percentagem de energia reproduzida com erro, facilitando a comparação com a literatura. Valores de ESR entre 0,1% a 1% correspondem a modelos de qualidade industrial nesta classe de problemas.

2.6.4 Optimizador: Adam

O optimizador adotado é o **Adam** (Kingma & Ba, 2014), com *learning rate* 5×10^{-3} . O Adam adapta o *learning rate* por parâmetro com base em médias móveis dos primeiros e segundos momentos dos gradientes.

2.6.5 Estratégia de Split de Dados

A divisão do *dataset* em conjuntos de treino e validação foi inicialmente concebida como **split temporal** (primeiros 80% do sinal capturado para treino, últimos 20% para validação). Esta opção é metodologicamente preferível em projetos com dados homogêneos no tempo, pois evita *temporal leakage* entre treino e validação.

Contudo, a sessão de captura efetuada produziu material heterogêneo no tempo (notas isoladas no início, acordes no fim), introduzindo *distribution shift* entre os primeiros 80% e os últimos 20%. Adotou-se, por isso, um **split aleatório por janelas**. Cada janela individual é classificada aleatoriamente para treino ou validação, ignorando posição temporal.

2.7 Aquisição e Pré-processamento de Dados

2.7.1 Sessão de Reamping

A captura dos pares *Dry/Wet* segue o protocolo de **reamping**:

1. Grava-se um sinal *Dry* (sinal direto de baixo elétrico) na DAW.
2. Reproduz-se esse mesmo sinal *Dry* digital, encaminhando-o para a entrada do pedal através de uma interface de áudio.

3. Grava-se o retorno do pedal (sinal *Wet*) na DAW, em paralelo com o sinal original.

Decisão de engenharia documentada: o pedal é mantido fisicamente na cadeia mesmo durante a captura do *Dry* (com o *footswitch* em *bypass*). Esta decisão anula impedâncias e capacitâncias parasitas comuns aos dois *takes* — a rede aprende **apenas** o efeito do circuito ativo, e não diferenças residuais introduzidas pelos cabos e conectores. Sem esta precaução, a rede aprenderia, indistintamente, o efeito do pedal e o ruído de impedância, comprometendo a generalização.

2.7.2 Alinhamento por Correlação Cruzada

Após a captura, os ficheiros *Dry* e *Wet* não estão tipicamente alinhados *sample-accurate*: a cadeia de *reamping* introduz uma latência (positiva ou negativa) imprevisível, agravada por mecanismos de *Plugin Delay Compensation* da DAW que nem sempre acertam. O treino exige alinhamento perfeito - cada amostra de saída tem de corresponder à amostra de entrada que a originou - pelo que se aplica um algoritmo de alinhamento.

O algoritmo adotado é a **correlação cruzada** entre os dois sinais. A correlação cruzada ($x * y$)[k] mede a semelhança entre x e uma versão de y deslocada de k amostras; o k que maximiza esta correlação corresponde ao deslocamento que melhor alinha os dois sinais. Após determinar k , compensa-se o desalinhamento cortando o número adequado de amostras do início de um dos ficheiros.

Decisão de qual ficheiro cortar. O sinal do deslocamento determina o ficheiro a cortar:

- $k > 0$ (*Wet* atrasado): cortar k amostras do início do *Wet*.
- $k < 0$ (*Wet* adiantado, sintoma típico de sobre-compensação de PDC): cortar k amostras do início do *Dry*.

A escolha contraintuitiva no caso negativo (cortar do *Dry* quando o *Wet* está adiantado) deriva de uma observação simples: para alinhar dois sinais quando um está adiantado, é necessário "atrasar" o adiantado — operação que se concretiza cortando do início do **outro** sinal.

Após o corte, valida-se o alinhamento recalculando a correlação cruzada e confirmando que o pico ocorre em $k=0$.

2.8 Inferência em Tempo Real

2.8.1 Restrições da Audio Thread

A *audio thread* da DAW *host* invoca a função `processBlock` do *plugin* em intervalos periódicos, fornecendo um buffer de N amostras por canal (tipicamente 64–512

amostras) e exigindo a saída processada antes do próximo intervalo. O período é dado por N / fs (e.g. $\sim 1,3$ ms para $N=64$ a 48 kHz), e qualquer falha em cumprir este prazo resulta em *dropouts* audíveis.

Sob estas restrições, a `processBlock` deve:

- **Não alocar memória dinâmica.** `malloc/new` introduzem latência imprevisível (eventualmente milissegundos em sistemas com fragmentação).
- **Não fazer I/O.** Leituras/escritas em disco ou rede são incompatíveis com a janela de tempo disponível.
- **Não bloquear em sincronização.** *Mutexes*, *spin-locks* ou *condition variables* podem bloquear indefinidamente.
- **Ser determinística.** Caminhos de execução com variação de tempo (e.g. *garbage collection*) são inadmissíveis.

2.8.2 Compile-time API e Inferência Sample-by-Sample

Para cumprir estas restrições, a RTNeural é utilizada na sua **API compile-time**, com a topologia da rede fixada nos *templates*:

```
RTNeural::ModelT<float, 1, 1,  
    RTNeural::LSTMLayerT<float, 1, 24>,  
    RTNeural::DenseT<float, 24, 1>> model;
```

Os dois primeiros parâmetros (1, 1) são as dimensões globais de entrada e saída do modelo; os parâmetros seguintes são as *layers*. Esta forma de instanciação permite ao compilador (a) fazer *inlining* agressivo das operações; (b) eliminar despachos virtuais; (c) alocar todos os buffers internos em memória estática (*stack* ou *.bss*), eliminando alocações em *runtime*. O custo é a perda de flexibilidade: alterar a arquitetura (e.g. mudar hidden de 24 para 32) implica recompilar o *plugin*. Esta inflexibilidade é um custo aceitável dado que o foco do MVP é um único modelo.

A inferência opera **amostra a amostra**: a LSTM mantém estado entre invocações sucessivas, e a *audio thread* exige cada amostra processada **em ordem**. O processamento de blocos itera pelas amostras do buffer e invoca `model.forward()` para cada uma:

```
for (int n = 0; n < numSamples; ++n)  
    output[n] = model.forward(input[n]);
```

2.8.3 Embedding de Pesos via JUCE BinaryData

Os pesos da rede (`model.json`), são embebidos diretamente no binário do *plugin* via mecanismo `BinaryData` da JUCE. A justificação é a portabilidade: o utilizador final não precisa de gerir ficheiros externos, atualizações do modelo são entregues como nova versão do *plugin*.

Restrição associada: o *build* em modo **Release** é essencial. O modo Debug, sem otimizações, é catastroficamente lento devido à natureza *template-pesada* da Eigen, causando *dropouts* mesmo com modelos pequenos. Esta restrição é registada na documentação de *build* e nos *headers* do *plugin*.

2.9 Estrutura de Dados e Classes

2.9.1 Pares Dry/Wet

No pipeline de treino, os pares *Dry/Wet* são representados por uma classe `AudioPairDataset` (extendendo `torch.utils.data.Dataset`) que:

- Carrega os dois ficheiros WAV em memória uma única vez na construção.
- Indexa janelas **não-sobrepostas** de comprimento fixo (2048 amostras), com $n_windows = total_samples // window_size$.
- Devolve, para cada índice, um par (*dry_window*, *wet_window*) em *tensors* de PyTorch.

Esta abstração isola o resto do *pipeline* dos detalhes de I/O e segmentação, e integra-se naturalmente com `torch.utils.data.DataLoader` para criação de *mini-batches*.

2.9.2 Classes Principais do Plugin

O *plugin* segue o padrão idiomático JUCE, com três classes principais:

- `AlphaOmicronProcessor` (extende `juce::AudioProcessor`): núcleo do *plugin*. Implementa `processBlock`, gere o ciclo de vida (preparação, processamento, libertação), expõe os parâmetros à DAW via `AudioProcessorValueTreeState` e contém duas instâncias do `NeuralModel` (uma por canal).
- `AlphaOmicronEditor` (extende `juce::AudioProcessorEditor`): interface gráfica. Cria os *widgets* (*toggle* de *Bypass*, *slider* de *Output Volume*) e liga-os aos parâmetros via `SliderAttachment` / `ButtonAttachment`.
- `NeuralModel`: encapsula a `RTNeural::ModelT` com a topologia *compile-time*. Expõe métodos `prepare()`, `reset()` e `processSample(float)`.

2.10 Interface do Utilizador

2.10.1 Plugin

A interface do *plugin* é deliberadamente minimalista, refletindo o âmbito MVP definido na proposta. Expõe dois controlos:

- **Bypass** (*toggle* binário): liga/desliga o processamento neural. Quando ativo (*bypass on*), o sinal de entrada é encaminhado para a saída sem inferência da rede mas com o ganho de saída aplicado, permitindo comparação A/B sem descontinuidade de nível.
- **Output Volume** (*slider* rotativo em dB): aplica um ganho de saída ao sinal processado, na gama de **-24 a +6 dB**, com *default* de 0 dB e passo de 0,01 dB. O *slider* é renderizado como *knob* rotativo (Slider::RotaryVerticalDrag), atendendo ao requisito não-funcional RNFO2 (controles rotativos intuitivos).

Ambos os controles são expostos à DAW através do mecanismo AudioProcessorValueTreeState, garantindo automação, recuperação de estado entre sessões e sincronização entre a GUI e o motor de áudio. Esta integração é idiomática JUCE e dispensa código *ad hoc* de gestão de parâmetros.

2.10.2 WebApp ABX

A WebApp implementa o protocolo experimental ABX, comumente utilizado em estudos psicoacústicos. Em cada *trial*, o participante:

1. Ouve a amostra **A** (uma das duas alternativas — pedal real ou *plugin*).
2. Ouve a amostra **B** (a outra alternativa).
3. Ouve a amostra **X** (sorteada aleatoriamente entre A e B).
4. Indica se X corresponde a A ou a B.

A submissão da resposta regista, para cada *trial*: identificador do participante (anónimo), identificadores das amostras A, B e X, resposta do utilizador, *timestamp* e tempo de resposta. A aleatorização das amostras em cada *trial* elimina viés de ordem; o anonimato dispensa procedimentos éticos formais para tratamento de dados pessoais.

Após um 12 *trials*, a aplicação apresenta ao participante o seu resultado individual e submete os dados ao *backend* de persistência. A análise estatística é executada *offline* sobre o conjunto consolidado de respostas.

Capítulo 3 - Implementação

Este capítulo reporta o estado de implementação do projeto à data de submissão do relatório intermédio. Começa pela decomposição do trabalho em tarefas e pelas suas precedências, apresenta o calendário detalhado comparando o plano com a execução efetiva, e descreve por ordem cronológica o estado de cada uma das frentes de trabalho.

3.1 Decomposição do Trabalho

O projeto foi decomposto em oito fases técnicas no caminho crítico e duas frentes paralelas de interface - uma na DAW (UI do *plugin*) e outra na *web* (WebApp ABX). A Tabela 3.1 lista as tarefas, as suas precedências.

Tabela 3.1 — Decomposição do trabalho, precedências e branches Git

ID	Tarefa	Precedências
F1	Setup do repositório e do toolchain	—
F2	Boilerplate do plugin JUCE com APVTS	F1
P1	UI do plugin (Bypass, Output Volume)	F2
P2	WebApp ABX (frente paralela)	F1
F3	Captura de pares Dry/Wet	F1
F4	Alinhamento Dry/Wet	F3
F5	Pipeline PyTorch e validação sintética	F1
F6	Treino com dados reais	F4, F5
F7	Integração RTNeural e inferência end-to-end	F2, F5
F8	Integração do modelo treinado	F6, F7
F9	Otimização, recaptura e regressão Py $\leftrightarrow$ C++	F8
F10	Recolha pública de respostas ABX	P2 + backend
F11	Análise estatística e redação final	F10

Caminho crítico. A sequência F1 $\rightarrow$ F2 $\rightarrow$ F7 $\rightarrow$ F8 $\rightarrow$ F11 é o caminho crítico do *plugin*; o ramo F1 $\rightarrow$ F3 $\rightarrow$ F4 $\rightarrow$ F6 $\rightarrow$ F8 é o caminho crítico dos dados, com F8 a depender de ambos.

As frentes paralelas P1 e P2 não atrasam diretamente a entrega final desde que P2 esteja publicada e com *backend* a tempo de viabilizar F10.

3.2 Calendário Detalhado: Planeado vs. Executado

A Tabela 3.2 compara o calendário planeado na proposta com a execução efetiva.

Tabela 3.2 — Calendário planeado vs. executado

Tarefa	Plano (Sem.)	Exec. (Sem.)	Estado	Observação
F1 — Setup	1–2	1	Concluído	Conforme plano.
F2 — Boilerplate JUCE	1–2	4	Concluído	Atrasado por preparação teórica nas Sem. 3–4.
P1 — UI do plugin	13	4	Concluído	Antecipado em ~9 semanas.
F3 — Captura Dry/Wet	4	6	Concluído	Atrasado por bloqueio de hardware
P2 — WebApp ABX	9–10	5	Concluído	Antecipado; submetido como E-Fólio A do LSSW.
F4 — Alinhamento	5–6	7	Concluído	Encadeado com F3.
F5 — Pipeline (sintético)	5	6–7	Concluído	Conforme plano.
F6 — Treino dados reais	6	7	Concluído	Conforme plano.
F7 — Integração RTNeural	6	7	Concluído	Conforme plano.
F8 — Modelo real no plugin	6	7	Concluído	Conforme plano.
Demo interna	7	7	Concluído	Marco cumprido.
F9 — Otimização e regressão	9–10	8–10	A iniciar	—
F10 — Recolha pública ABX	11–13	11–13	Pendente	Depende de backend.

F11 — Análise e redação final	13–16	13–16	Pendente	—
-------------------------------	-------	-------	----------	---

3.3 Estado de Implementação por Fase

As subsecções seguintes descrevem o trabalho realizado em cada fase concluída, os artefactos produzidos e os resultados quantitativos quando aplicável. Os blocos de código, *outputs* extensos e capturas de ecrã são remetidos para os anexos referenciados; o repositório público (https://github.com/ZzOne/Emulacao_de_Pedal_Analogico_com_Machine_Learning) é a fonte autoritativa do código.

3.3.1 Setup e Boilerplate JUCE (F1, F2, P1)

O plugin foi gerado a partir do template de referência da JUCE (TheAudioProgrammer), em C++20, com gestão de dependências via CMake + CPM e formatos compilados em VST3, AU e Standalone.

As três classes principais — AlphaOmicronProcessor, AlphaOmicronEditor e NeuralModel — implementam o desenho descrito em 2.9.3. Os parâmetros expostos à DAW via APVTS são bypass (bool, default false) e outputVolume (float, gama -24 a +6 dB, default 0 dB, atendendo aos requisitos não-funcionais relevantes). A UI foi implementada no branch juce_ui.

3.3.2 WebApp ABX (P2)

A WebApp foi implementada no branch webapp_abx num sprint concentrado em 18 de Abril. Foi submetida como E-Fólio A do Laboratório de Sistemas e Serviços Web a 20 de Abril e cumpre integralmente o protótipo client-side do MVP do projeto.

A aplicação tem cinco páginas HTML interligadas (index, project, instructions, test, results) e seis módulos JavaScript com separação clara entre lógica (motor ABX, estatística, síntese áudio, storage) e DOM (controladores de página). Os módulos seguem o padrão IIFE expostos em window. A persistência é estritamente client-side via localStorage, com retoma de sessão pendente. A análise estatística inclui PMF binomial em log-space, p-value unilateral para $H_0: p = 0,5$, e cálculo de d' pelo algoritmo de Beasley-Springer-Moro com correção de Hautus para proporções extremas.

3.3.3 Captura, Alinhamento e Pipeline de Treino (F3, F4, F5)

Captura (F3). Realizada na Semana 6 com a cadeia descrita em 2.7.1. A sessão estava planeada para a Semana 4 mas foi adiada pela perda da fonte de alimentação do pedal - bloqueio registado no changelog da Sem. 4 e resolvido com a aquisição de uma fonte substituta. Output: 5 minutos e 24 segundos de pares Dry/Wet a 48 kHz / 24 bit, com duas limitações documentadas para mitigação em F9 - níveis baixos (pico do Dry a -11 dBFS, do Wet a -22 dBFS) e heterogeneidade temporal do material.

Alinhamento (F4). Implementado em dois scripts Python (`align_dry_wet.py` aplica a compensação, `check_alignment.py` valida-a por correlação cruzada). O lag medido foi de -241 amostras ($\approx -5,02$ ms a 48 kHz), sintoma típico de sobre-compensação de Plugin Delay Compensation do Reaper com hardware externo. Após o corte, a correlação cruzada apresenta pico em $k = 0$. Durante o desenvolvimento detetou-se um bug silencioso no backend TorchCodec do `torchaudio.save` (ignorava operações de slicing sem qualquer indicação de erro), contornado pela substituição por `soundfile.sf.write`.

Pipeline PyTorch (F5). Quatro módulos em `src/training/` - `dataset.py` (AudioPairDataset com janelas não-sobrepostas e split aleatório por janelas), `model.py` (LSTMemulator), `loss.py` (ESRLoss) e `train.py` (loop com TBPTT). A validação sintética com `dry` = ruído gaussiano e `wet` = $\tanh(50 \times \text{dry})$ convergiu para $\text{ESR} \approx 0,005$ ($\sim 0,5\%$ de erro relativo), confirmando a correção do pipeline antes da aplicação a dados reais.

3.3.4 Treino com Dados Reais (F6)

Esta fase exigiu uma correção metodológica intermédia, registada por revelar uma armadilha estrutural relevante. A primeira tentativa aplicou split temporal estrito (primeiros 80% para treino, últimos 20% para validação), mas a loss de validação apresentou comportamento errático com valores acima de 1,0 — diagnóstico: distribution shift causado pela heterogeneidade temporal do material capturado (notas isoladas no início, acordes no fim). A correção consistiu em adotar split aleatório por janelas via `split_dataset_random(..., seed=42)`.

Após a correção, duas configurações foram treinadas:

Configuração	Epochs	ESR final	Observação
<code>hidden_size = 8</code>	30	$\approx 0,20$	Plateau; modelo subdimensionado
<code>hidden_size = 24</code>	50	$\approx 0,030$	3% de erro relativo, dentro da gama da literatura

Observou-se ainda um padrão de convergência tardia (grokking) no modelo `hidden=24`: ~ 45 epochs em plateau de $\text{ESR} \approx 0,20$ seguidas de colapso para $\approx 0,03$ nas três últimas. O fenómeno é conhecido na literatura de redes recorrentes e corresponde à passagem por uma configuração de pesos onde múltiplas componentes se reorganizam abruptamente - não é um bug da implementação.

3.3.5 Integração RTNeural e Modelo Treinado (F7, F8)

Integração RTNeural (F7). A classe `NeuralModel` em `src/dsp/` encapsula a `RTNeural::ModelT` com a topologia compile-time descrita em §2.8.2 (`HIDDENSIZE = 24`). O carregamento dos pesos ocorre uma vez em `prepare()` em três passos: extração do JSON embebido via `JUCE BinaryData`, parse para `nlohmann::json`, e mapeamento direto via `RTNeural::torch_helpers::loadLSTM` e `loadDense`. Inclui guard bypass-safe que devolve a amostra de entrada se a inicialização falhar.

Integração do modelo treinado (F8). O script `export.py` serializa o `state_dict` PyTorch no formato esperado pelos helpers da RTNeural — ficheiro JSON de ~80 KB com 2617 parâmetros em float32. A topologia já tinha sido fixada em `HIDDENSIZE = 24` no commit anterior, pelo que a integração consistiu apenas em substituir o `model.json` em `src/alpha_omicron_plugin/resources/` e recompilar em modo Release.

Validação informal. O plugin opera no Reaper com buffer de 256 amostras a 48 kHz, sem dropouts e com som audívelmente o do pedal. Em testes informais de comparação A/B, senti dificuldade subjetiva em distinguir o plugin do pedal real.

3.4 Limitações Reconhecidas

Limitações identificadas durante a execução, com a respetiva mitigação prevista:

- Níveis de captura subaproveitados (~13 bits efetivos). Mitigação (F9): recaptura com calibração para -3 a -1 dBFS de pico.
- Heterogeneidade temporal do material capturado, que forçou o split aleatório com temporal leakage. Mitigação (F9): recaptura com material distribuído homogeneamente, viabilizando split temporal estrito.

Capítulo 4 - Testes

<Exemplifique o funcionamento normal do sistema, com capturas de ecrãs, relacionando a execução com a especificação, de modo a garantir que todos os elementos da especificação são exemplificados>

<Apresente os testes de funcionalidade, tanto unitários como de todo o sistema, que comprovem o bom funcionamento deste>

<Apresente testes de desempenho>

<Apresente testes de escalabilidade>

<Apresente resultados obtidos>

Capítulo 5 - Conclusões

<Reflexões e conclusões sobre os resultados face aos objetivos pretendidos>

<Identificar as dificuldades e limitações do trabalho>

<Identificar possíveis melhorias>

<Identificar trabalho futuro que pode ser feito na sequência do projeto>

Bibliografia

<Inclua a lista de bibliografia utilizada, sejam artigos, livros, hiperligações, manuais, etc. Utilize o formato APA. Nas hiperligações, deve sempre referir a data da última visita. Ordene alfabeticamente e no corpo do relatório, utilize o padrão APA (nome de autor(es), ano)>

Anexo I - xxx

<Coloque quantos anexos forem necessários com os elementos produzidos no projeto, seja código ou outros. >